arXiv:2609.23886v1 [cs.CL] 20 Sep 2026

this-that-model-1.0

A typed decision model that decides in 30 ms, for a millionth of a cent

Zehua Cheng[1,2] Wei Dai[2] Jiahao Sun[2]

[1]University of Oxford [2]FLock.io

ai@flock.io

Abstract

Software delegates more of its branches to models every year: which queue a ticket enters, whether a command is safe to run, whether a claim clears without a person. What the program needs back is not prose. It is one of n declared options and a number it can threshold. Today that costs a round trip to a frontier model — hundreds of milliseconds, a per-token bill, and a parser — for a question that is usually a conjunction of three clauses.

`this-that-model-1.0` is a 2B-parameter typed decision model. Its answer is read directly from the hidden state at a designated position and restricted to the option set the caller declared, so no text is generated, nothing can be malformed, and every question in a request is answered in the *same* forward pass.

It decides in 30.9 ms on one laptop GPU, and generates *zero* output tokens doing it. Both halves matter. At 30.9 ms a decision costs about what a database index lookup costs, so it can go inside a loop, inside a retry path, inside a validator that runs on every request — places where a 8758 ms API call is not an option at any price. And the zero is not a small number, it is the absence of one: the hosted systems that answer these questions well spend between 21 and 212 generated tokens per question thinking first, and are billed for every one. `this-that-model-1.0` sustains 32 decisions per second on one consumer GPU and never lets the state leave the machine. This paper describes how that is built — the typed head, a prompt layout whose schema prefix is computed once and reused across states, a training objective that is a proper scoring rule estimated without bias from single sampled outcomes, and a deployment scheme in which the entire adaptation is one scalar — and then measures it.

On a third party's recorded cohort of 68 decision questions, on their inputs and their wording, it scores 0.941 with a Brier score of 0.042, against 0.765 and 0.133 for `Jev`, the hosted service, on the same items. One pass of our 42-family internal suite takes 32 seconds and $0.000217 of electricity; the most accurate hosted model we measured needs 155.2 minutes and $10.636. We also report where it loses. On the branch-shaped families the product is for it scores 0.818, against 0.837 to 0.968 for the hosted models; on multi-step arithmetic, which a single forward pass cannot carry intermediate results through, it scores 0.560 against their 0.98 to 1.00. And a targeted second training round improved the five task families it was written for and transferred to none of the other 13.

1 The smart if-statement

A program reaches a branch it cannot express in code. Is this refund within policy? Does this output satisfy the instruction it was given? Is this shell command safe to run unattended? The

condition is real, it is decidable, and no regular expression will decide it.

The common answer is to call a large language model and parse what comes back. That works, and it is expensive in a specific way. The model must produce tokens, so the cost is a decoding loop rather than a single evaluation; the output is free-form, so the caller must parse it and handle the case where parsing fails; and the call leaves the machine, so the state being judged leaves with it. For a branch on a request path, the accuracy of the frontier model is rarely what hurts. The 8758 ms and the invoice are.

The parsing problem deserves more than a clause, because it is the part that survives into production. A generative model asked for one of two options can return the option, the option inside a sentence, a refusal, a markdown fence, an empty string when a reasoning budget was spent before the answer arrived, or a plausible word that is not on the list. Each of these is a distinct branch in the caller, and the caller must pick a behaviour for the case where none of them parse: fail open, fail closed, or retry. That decision is a policy choice about correctness, and it is usually made once, quickly, in an exception handler. We hit this from the measurement side while writing this paper — a frontier model appeared to score 0.208 on a task it answers correctly 0.909 of the time, because a truncated reply had been scored as a wrong answer rather than as no answer — and the general lesson is that "the model returned nothing" and "the model was wrong" are different events that a text interface makes hard to keep apart.

A *typed decision model* removes the decoding loop and the parser with it. The caller declares the answer type up front — a choice among named options, an ordered score, or a boolean — and the model returns a probability distribution over exactly that set. Nothing is sampled; one forward pass produces the hidden state at each answer position, and the distribution is a softmax over the option labels and nothing else. The set of possible outputs is the set of declared options, so a malformed answer is not unlikely, it is unrepresentable, and the branches above collapse into a lookup. This paper is about building one that is small enough to run beside the application and good enough to be trusted with the branch.

Two things follow that are not obvious from "it returns a letter". The first is that the number attached to the answer is a usable probability rather than a token score. Trained against a strictly proper scoring rule (Section 3.3), the model has no way to improve its loss except by reporting what it believes, which is what lets a caller put a threshold on the output and route the uncertain cases somewhere else. On the one family in our measurements whose true answer is a computed probability rather than a label, the model lands at 0.750 against a ceiling of 0.746 that no predictor can exceed, while most hosted endpoints expose no probability at all and every one that does is worse than a constant. The second is that N questions about one state cost one pass rather than N, because no answer is written back into the prompt and the decisions are therefore conditionally independent given the input — so a decision point that asks four things about a request is not four times the price of one.

The number that decides whether such a thing is usable is the latency, and ours is 30.9 ms. A branch that costs 30.9 ms can be evaluated on every request without a budget conversation; a branch that costs 8758 ms cannot be evaluated on every request at all, and the usual response is to sample it, to cache aggressively, or to give up and write a worse rule in code. One laptop GPU sustains 32 decisions per second, which is what an application tier would otherwise buy from an API for $10.636 per thousand questions. The model is 1.88B parameters, it runs on a card that is already in the machine, and because it runs there the state being judged never leaves — which for a refund, a shell command or a customer record is frequently the binding constraint rather than the price.

We should say plainly what this is not. It is not a claim to be more accurate than a frontier model: on the harder half of our suite it is not, and on a benchmark we built and released for this

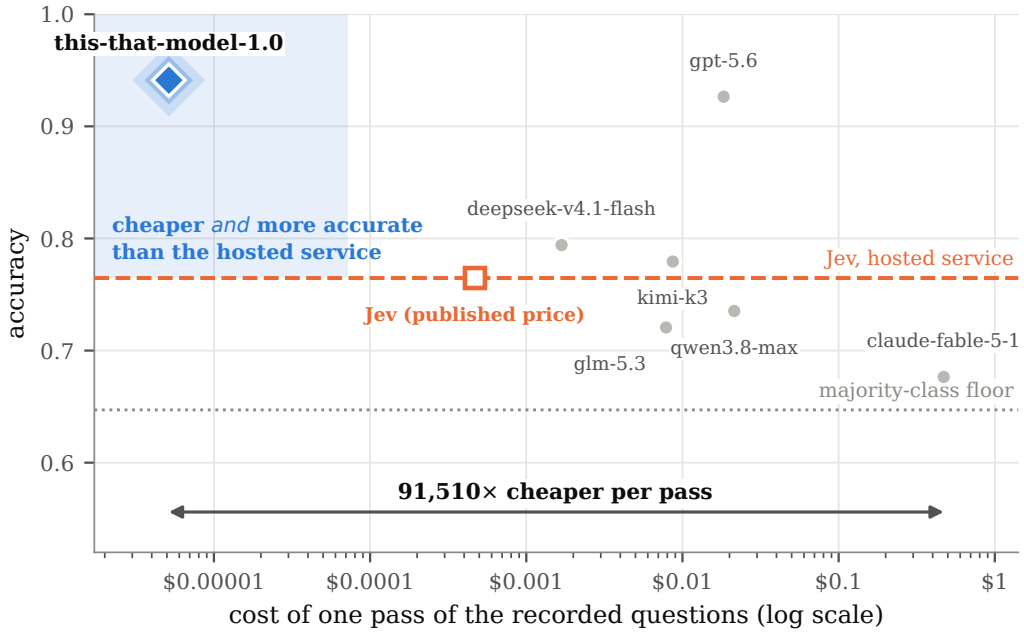


Figure 1. Accuracy against what one pass of the recorded questions costs. Cost spans five orders of magnitude; accuracy spans twenty-five points, and one system is alone in the shaded corner. Two caveats the chart cannot carry. The hollow square is `Jev` priced at its *published* rate of $0.042 per million input tokens with output free, applied to the token count we measured — its accuracy is ours to measure, its cost is theirs to state. And our own cost is electricity at 80 W on a card already bought, which is a different kind of number from a price that has to cover serving and margin; the honest reading of our position against the square is one order of magnitude, not five.

paper it is the least accurate system we measured (Section 4.3). It is a claim about the cost of a decision, and about a shape of output that a program can rely on. The honest summary of the trade is that a frontier model is a better judge and a worse component, and that a great many branches in real systems need a component.

Contributions. (i) A typed answer head that makes malformed output impossible by construction and answers N questions about one state in a single forward pass (Section 3.1). (ii) A prompt layout whose cost is amortised over states rather than paid per question (Section 3.2). (iii) A training objective that is a strictly proper scoring rule and can be estimated without bias from a single observed outcome per question, using a paired control variate (Section 3.3). (iv) A deployment scheme in which the entire adaptation is a single scalar, so rollback is exact rather than approximate (Section 3.4). (v) Measurements against `Jev`, a hosted commercial service, on inputs we did not choose, and against 9 hosted frontier models on identical questions whose answer is a computed *distribution* rather than a sampled label (Section 4).

2 Related work

Typed and constrained model outputs. Structured decoding — grammars, JSON schemas, regular-expression constraints [10, 25] — makes a generative model's output parseable, but it remains a decoding loop, and the constraint is applied to the tokens rather than to the hypothesis space. The typed-decision formulation instead reads the answer from a hidden state and normalises over the declared options, which removes the loop rather than disciplining it. The commercial

"System One" line of models [1], named for the fast automatic mode of reasoning [15], popularised this interface for application developers; `decider-2b` is a publicly released 2B typed-decision checkpoint and inference stack under Apache-2.0: the architecture family our model belongs to, and the source of the wire format that Section 3.1 formalises. NanoJev [22] is an open 0.6B reproduction of the same interface under the MIT licence; it contributes the environment simulator and the recorded hosted-service cohort we measure against in Section 4, and its own checkpoint appears in Table 1 as a baseline.

Proper scoring rules and calibration. That a decision model's probability must be usable, not merely rankable, is old: strictly proper scoring rules [11], of which the logarithmic and the Brier score [2] are the two canonical members, are optimised uniquely at the true conditional distribution. Calibration is usually measured after the fact, by binning predictions against observed frequencies [12, 20, 21], a procedure whose estimator is itself sensitive to the binning [21]. What is less common is evaluating against a *known* distribution rather than a sampled label, as simulation-based calibration does for posterior inference [6, 24]; Section 4 does that where the environment makes it possible, and Equation (13) says exactly what is lost when it is not.

Weight-space adaptation. Interpolating between a base and a fine-tuned checkpoint along their difference vector is a standard tool for trading specialisation against generality [14, 26, 27], and one answer among several to the forgetting that full fine-tuning induces [18]; parameter-efficient adaptation [13] is another, which we did not use because the budget allowed the full fine-tune. We use it for an operational property rather than for accuracy — the zero setting is an exact identity, so a rollback needs no requalification (Section 3.4).

Linear-attention backbones. Delta-rule and related linear-attention layers [16, 29] replace the quadratic attention matrix with a fixed-size recurrent state, giving $O(L)$ cost in sequence length. Hybrids that interleave a minority of full-attention layers recover most of the quality [28]. This is background for us, not contribution, and it is why a 32k-token state is affordable at all.

3 Method

3.1 The typed answer head

Let x be the rendered request: a state S (a string, or any JSON value serialised compactly) and questions $q_1, \dots, q_N$, where question k declares an ordered option set $A_k = (o_{k,1}, \dots, o_{k,n_k})$. The request is rendered so that each question ends at a designated *answer slot* — a fixed token position s_k — and the model is run once:

$$H \;=\; f_\theta(x) \in \mathbb{R}^{L \times d}, \qquad h_k \;=\; H_{s_k}. \tag{1}$$

Each option is assigned a label token whose embedding row in the unembedding matrix $W \in \mathbb{R}^{|V| \times d}$ is w_ℓ; write $\ell(k, j)$ for the label token of option j of question k. The answer distribution is the softmax over *only* those rows:

$$p_k(j \mid x) \;=\; \frac{\exp\big(\langle w_{\ell(k,j)},\, h_k \rangle / \tau\big)}{\sum_{j'=1}^{n_k} \exp\big(\langle w_{\ell(k,j')},\, h_k \rangle / \tau\big)}, \qquad j = 1, \dots, n_k. \tag{2}$$

Three properties follow directly from (2) and are the reason for the design.

The output cannot be malformed. The support of p_k is A_k by construction. There is no parser and no retry path, because there is no string. A generative model can be constrained towards this behaviour by decoding rules, but the probability it assigns to the declared set is $\sum_j P(\text{option } j) < 1$ in general, and the residual mass has to be handled somewhere. Here it is identically zero.

The cost is one evaluation, not a loop. Generating T answer tokens costs T sequential forward passes and T round trips through the sampler. Equation (2) costs one pass and one inner product per option.

The N answers share that pass. No answer letter is ever written into the context, so slot k attends to the state and to all N questions, but never to another answer. The model therefore factorises the joint as

$$p(a_1, \ldots, a_N \mid x) \;=\; \prod_{k=1}^{N} p_k(a_k \mid x), \tag{3}$$

which is the conditional-independence assumption a caller wants when it asks N separate questions about one state. Answering them separately would cost N passes over $|S|$ tokens; answering them jointly costs one pass over $|S| + \sum_k |q_k| + N$.

Labels are chosen so that every option costs exactly one token. For $n_k \le 10$ the rendering is the familiar `(A)`…`(J)`; beyond that we use a table of 255 upper-case strings that the tokenizer maps to single ids, which keeps (2) a single gather regardless of how many options the caller declares.

3.2 Rendering: where the state goes

The obvious layout puts the state first and the questions after it. It has a defect at scale. In production the same *schema* — the questions and their option sets — is asked about many different states: one classifier, a million tickets. With the state first, nothing is shared between requests.

We therefore also support a schema-first layout, in which the token stream is

$$\underbrace{[\,q_1, A_1, \ldots, q_N, A_N\,]}_{\text{prefix } P\text{, depends only on the schema}} \;\Vert\; \underbrace{[\,S\,]}_{\text{the state}} \;\Vert\; \underbrace{[\,\text{slot}_1, \ldots, \text{slot}_N\,]}_{\text{answer slots}}. \tag{4}$$

The prefix is byte-identical for every state, so its attention keys and values — and, in the linear-attention layers, its recurrent state — are computed once and reused, which is the same sharing that prefix caching exploits in a serving stack [19]. Serving m states under one schema costs

$$C_{\text{schema-first}} \;=\; |P| \;+\; m\,(|S| + N) \qquad \text{instead of} \qquad C_{\text{state-first}} \;=\; m\,(|P| + |S| + N), \tag{5}$$

a saving of $(m-1)|P|$ tokens of prefill. Schemas with many options are exactly the case where $|P|$ is large, so the saving grows with the thing that would otherwise hurt. Training mixes the two layouts so that one checkpoint serves both.

Why long states are affordable. The backbone is a hybrid: most layers are delta-rule linear-attention layers, which carry a fixed-size matrix state S_t updated per token,

$$S_t \;=\; S_{t-1}\big(I - \beta_t\, k_t k_t^{\top}\big) \;+\; \beta_t\, v_t k_t^{\top}, \qquad o_t \;=\; S_t\, q_t, \tag{6}$$

interleaved with a minority of full-attention layers. The linear layers cost $O(L)$ in sequence length rather than $O(L^2)$ and keep a constant amount of state per token, which is what makes a 32k-token state a routine request rather than an incident. We did not design this backbone; we mention it because it is the reason the numbers in Figure 4 are shaped the way they are.

3.3 Training against a proper scoring rule

A decision model is useful only if the number it returns can be thresholded, so the training objective must be a *proper scoring rule*: one whose expected value is optimised, uniquely, by reporting the true conditional distribution. Cross-entropy is one. The Brier score,

$$\mathrm{B}(p, y) \;=\; \|p - e_y\|^2, \tag{7}$$

is another, and it is the one that matches how the output is consumed — a squared penalty on the probability itself rather than on its logarithm, so it does not pay unboundedly for rare confident errors. We train on a convex combination of the two.

Learning from what actually happened. Some of our supervision is not a label but an *outcome*: an action was taken in a stochastic environment and something happened. For these we want to optimise the expected Brier reward

$$R(p) \;=\; \mathbb{E}_Y\big[\,2\,p_Y - \|p\|^2\,\big] \qquad \big(= -\,\mathbb{E}_Y\,\mathrm{B}(p, Y) + \text{const}\big), \tag{8}$$

where Y is the option the environment rewarded. Given M draws $a_1, \dots, a_M \sim p$ and one observed outcome y, the quantity

$$\widehat{R} \;=\; \frac{2}{M}\sum_{i=1}^{M} \mathbf{1}[a_i = y] \;-\; \frac{2}{M(M-1)}\sum_{i<j} \mathbf{1}[a_i = a_j] \tag{9}$$

is unbiased for $R(p)$: the first term estimates $2p_y$, and the second is the U-statistic that estimates $\|p\|^2$ without the bias that $\frac{1}{M}\sum_i p_{a_i}$ would carry. The gradient estimator assigns each draw its own credit r_i (its terms of (9)) and subtracts a paired control variate b_i formed by substituting p for the sampled indicators,

$$\nabla_\theta \widehat{R} \;=\; \sum_{i=1}^{M} \big(r_i - b_i\big)\,\nabla_\theta \log p(a_i), \qquad \mathbb{E}\big[b_i\,\nabla_\theta \log p(a_i)\big] = 0, \tag{10}$$

so the variance reduction costs nothing in bias. In practice $M = 32$ draws per question is enough that the estimator is quieter than the data.

Abstention. A decision system that may decline is a selective predictor [8, 9], and a caller that offers "none of the above" must not leak information by offering it. During training we insert an abstention option into a fraction of questions — with the correct answer removed in exactly the cases where abstaining is correct — so that the presence of the option is uninformative about whether it is the answer.

3.4 Shipping the adaptation as one number

Let θ_0 be the checkpoint we started from and $\theta_\star$ the checkpoint after training. We deploy not $\theta_\star$ but the family

$$\theta(\lambda) \;=\; \theta_0 \;+\; \lambda\,\Delta, \qquad \Delta \;=\; \theta_\star - \theta_0, \qquad \lambda \in [0, 1], \tag{11}$$

shipped as a single scalar alongside the two checkpoints. This is worth a paragraph for an operational reason rather than a scientific one: $\theta(0) = \theta_0$ *exactly*, bit for bit, because (11) is an identity and not an approximation. A rollback is therefore a configuration change with a proof attached, not a redeploy that has to be re-qualified, and intermediate λ gives a graded response when a regression is found in production. We ship $\lambda = 1$ and report the sensitivity of the headline numbers to λ in Section 4.

Table 1. The same 68 recorded questions over 17 states, sorted by accuracy.

system	accuracy	Brier ↓	NLL ↓	ms per question	cost of one pass ($)
majority-class baseline	0.647	—	—	—	—
`claude-fable-5-1`	0.676	—	—	2395	0.471
`glm-5.3`	0.721	0.204	0.601	819	0.008
`qwen3.8-max`	0.735	0.263	2.681	1014	0.021
`NanoJev-0.6B`	0.750	0.166	0.479	—	—
Jev	0.765	0.133	0.403	—	—
`kimi-k3`	0.779	0.143	0.430	989	0.009
`deepseek-v4.1-flash`	0.794	—	—	808	0.002
`gpt-5.6`	0.926	—	—	1180	0.018
`this-that-model-1.0`	**0.941**	**0.042**	**0.126**	**30.9**	**0.000014**

A dash in the Brier and NLL columns means the endpoint returns no token probabilities, so those quantities are not observable there — not that they are poor. The time column is the median *per-question* latency measured at the client. We do not report the wall clock of the whole pass, because that depends on how many requests our harness chose to keep in flight and is a property of the harness rather than of the system. Cost is for one pass of all 68 questions; for `this-that-model-1.0` it is electricity at 80 W and $0.30 per kWh. A further 3 frontier models answered every question correctly and are omitted: at 68 items they are saturated and rank nothing. `NanoJev-0.6B` is the open reproduction that made the recording; like `Jev` it was not timed by us, so its time and cost cells are blank rather than zero.

Fitting the work on one card. The adaptation is a full-parameter fine-tune of a 2B model on a single 16 GB laptop GPU, which is only possible with all of: 8-bit optimiser moments [7] (halving the 2× fp32 state per parameter that dominates the budget), gradient checkpointing, token-bucketed batching that keeps the number of distinct (B, T) shapes near twenty-four so kernels compile once, and a hard cap on the allocator so that an over-large batch raises instead of spilling into host memory. None of this is novel; all of it is load-bearing, and the absence of any one of them turns the run into a crash.

4 Measurements

4.1 Against `Jev`, on inputs neither of us chose

Vendor comparisons usually reduce to each side quoting its own benchmark. One exception is available. NanoJev [22], an open reproduction of the typed-decision interface, put 68 questions about local geometry across 17 states to the hosted *Jev* service [1] and published the whole exchange: the rendered inputs, the service's replies, the ground truth, and an input hash per item. Any system can therefore be placed on exactly those inputs, with exactly that wording, and the comparison is auditable by anyone who fetches the recording. Ground truth is computed from the environment rather than annotated, so the top of the scale is exact rather than nominal. Table 1 also carries NanoJev's own 0.6B model, which was trained on this style of question and is the strongest small open system we are aware of on it.

4.2 Whether the probability means anything

Accuracy cannot tell you whether a probability is honest, because a label carries no probability of its own. In an environment with a stochastic actuator it can. Let the intended action be $a^\star$, executed with reliability ρ and otherwise replaced by a uniform draw from the remaining actions. Then the

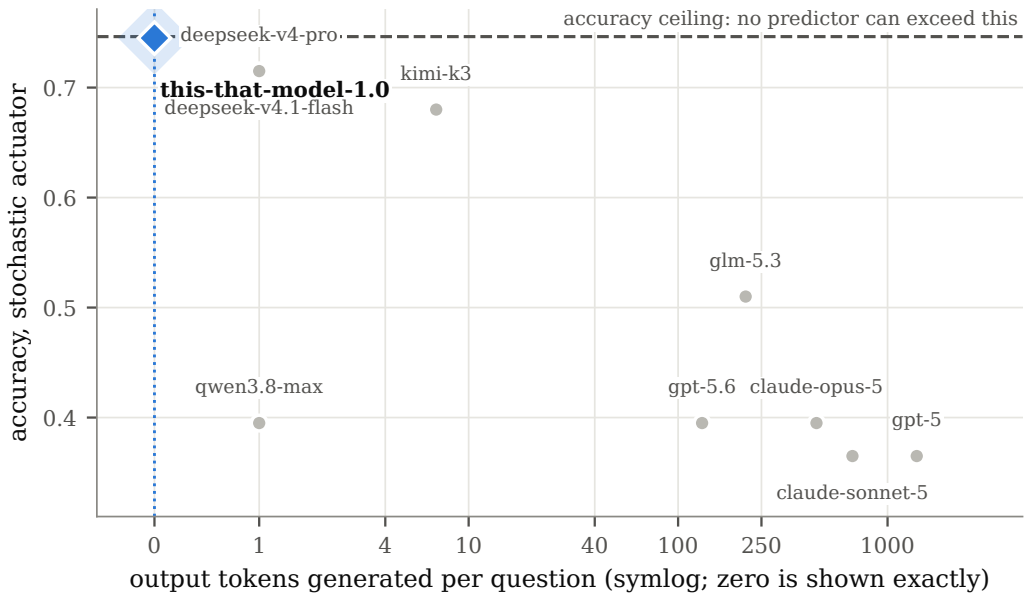


Figure 2. Accuracy on the stochastic-actuator family against the number of tokens each system generated before answering. The relationship runs the wrong way for a reader who expects test-time computation to help: the systems that generated the most tokens are the least accurate, and the three most accurate answer in one token or none. The environments make a noisy actuator safe more often than not, and a model that reasons its way to caution is reasoning away from the answer. `this-that-model-1.0` sits at zero tokens by construction — Equation (2) has no decoding step for tokens to be spent in — and on the accuracy ceiling.

probability that the executed move is collision-free is not estimated but derived:

$$q \;=\; \rho\,\mathbf{1}[\text{safe}(a^\star)] \;+\; \frac{1-\rho}{|A|-1}\sum_{a\neq a^\star}\mathbf{1}[\text{safe}(a)]. \tag{12}$$

The model trains on a single Bernoulli draw $Y \sim \text{Bern}(q)$ and is scored against q, which it never sees. We report $\text{q}L_2 = \mathbb{E}\|p-q\|^2$.

This is worth the trouble because sampled labels hide exactly this quantity. Decomposing the Brier score against the sampled outcome,

$$\mathbb{E}_Y\big[(p-Y)^2\big] \;=\; (p-q)^2 \;+\; q(1-q), \tag{13}$$

the second term depends on the environment and not on the model. It therefore cancels in *differences* between models — label-based comparisons are informative — but it does not cancel in levels, so no single Brier number on sampled data says how far a model is from the truth. Equation (12) closes that gap. A further consequence of (13) is a ceiling: no predictor can exceed $\mathbb{E}[\max(q,1-q)]$ accuracy on such labels, however good it is.

We report $\text{q}L_2$ beside two predictors that learn nothing: a constant equal to $\mathbb{E}[q]$, and the best predictor that reads ρ out of the prompt and ignores the environment entirely. The second is the one that matters — a model can look well calibrated while having learned only to copy a number out of its input (Table 2, Figure 3).

The decomposition is not a property of our model, and the honest way to show that is to run the identical ladder through every hosted system we have access to (Figure 5). The result is the same shape in every system that can do the task at all. Given the part the answer depends on, the curve is flat and near-perfect from 32×32 to 200×200. Given the whole map, it decays — and the

Table 2. 9 hosted systems and `this-that-model-1.0` on identical questions.

	stochastic actuator		local geometry		no	cost of
system	accuracy	qL_2 ↓	accuracy	qL_2 ↓	answer	one pass ($)
accuracy ceiling (computed)	0.746	—	1.000	—	—	—
always-yes baseline	0.703	—	—	—	—	—
constant predictor	—	0.0962	—	0.4934	—	—
`this-that-model-1.0`	**0.745**	**0.0248**	**1.000**	**0.0015**	**0**	**<0.001**
`deepseek-v4-pro`	0.715	—	0.520	—	0	0.043
`deepseek-v4.1-flash`	0.715	—	0.560	—	0	0.011
`kimi-k3`	0.680	0.1061	0.605	0.4808	0	0.059
`glm-5.3`	0.510	0.3064	0.985	0.0354	0	0.219
`claude-opus-5`	0.395	—	0.990	—	28	8.599
`gpt-5.6`	0.395	—	0.850	—	2	0.345
`qwen3.8-max`	0.395	0.7884	0.545	0.8794	0	0.138
`claude-sonnet-5`	0.365	—	0.960	—	88	1.763
`gpt-5`	0.365	—	1.000	—	120	2.964

The three rows above the rule are not systems: they are the best accuracy any predictor can reach on these labels, the score of answering "yes" every time, and the qL_2 of a constant. *No answer* counts replies that carried no usable letter even after the budget was raised to 4096 tokens; those are reported here rather than folded into the accuracy column, because returning nothing and answering wrongly are different events. A dash under qL_2 means the endpoint exposes no token probabilities at all.

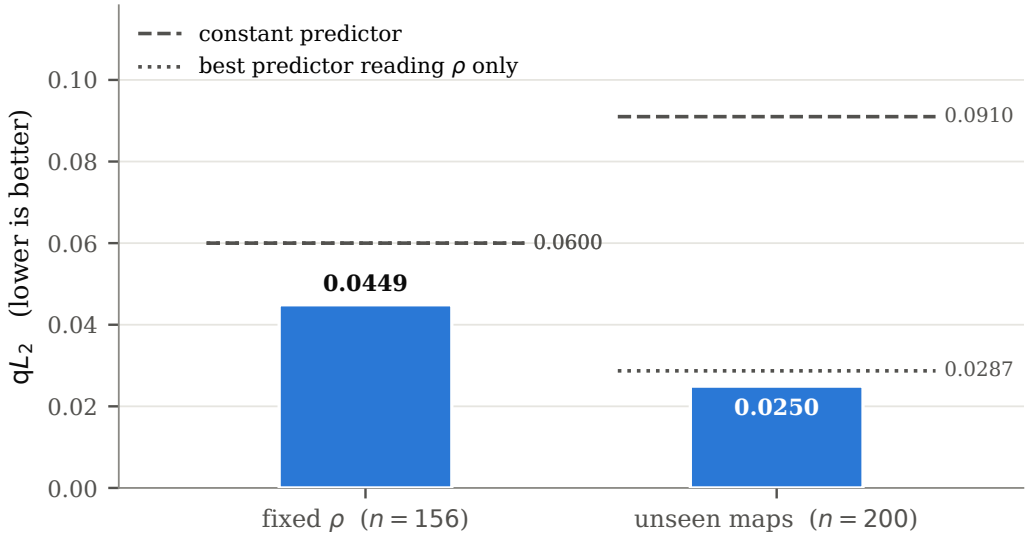


Figure 3. Squared distance to the computed answer distribution, against the two predictors that learn nothing. The bar is below both rules in both regimes; the point of drawing the rules is that a model can look calibrated by copying ρ out of its prompt, and this one is not doing that.

decay is not mostly wrong answers. It is silence: 19 % of the whole-map questions at the largest size went unanswered across the hosted systems, against 0 % of the windowed ones, each of those failures having spent a full 4096-token budget on a state of roughly 20,000 tokens and returned nothing. We count those as wrong, because a decision the caller never receives is not a decision.

The size of the effect is the part worth keeping. `claude-sonnet-5` answers 0.104 of the questions correctly when handed the whole 200 × 200 map and 0.979 when handed the window — the same model, the same questions, the same worlds, differing only in how much of the state it was given.

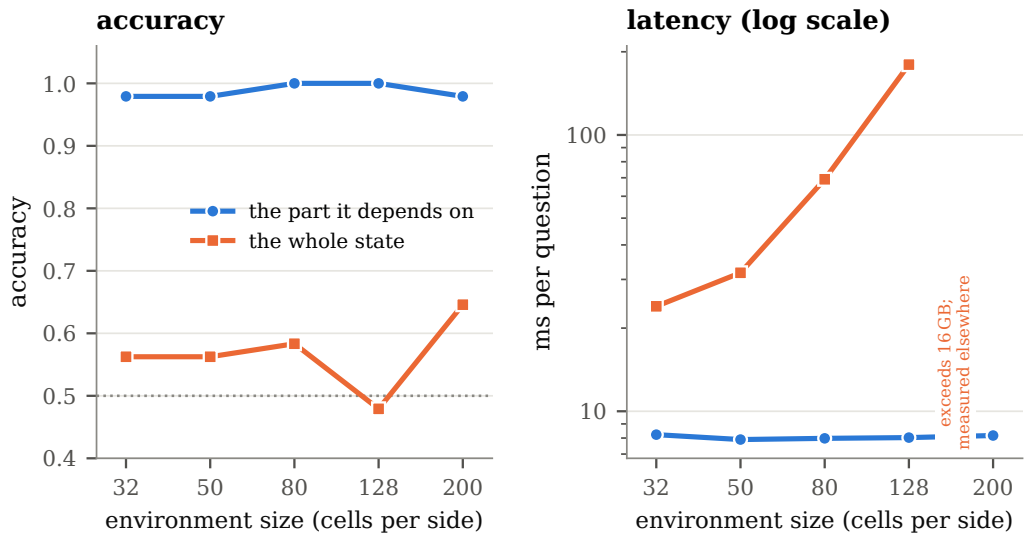


Figure 4. Two panels rather than two y-axes. Reading the part the answer depends on is flat in both accuracy and latency as the world grows; reading the whole map is worse at every size and costs an order of magnitude in time by 128 × 128. The whole-state input at 200 × 200 is 20,000 tokens, which does not fit the 16 GB card the latency panel was measured on; that rung's accuracy was measured later on an 80 GB card and its latency is therefore absent rather than merged in from another machine.

4 of the 10 hosted systems gain more than 0.2 from the window and none loses anything by it. The cheaper systems sit near chance under both renderings, which is what tells us the ladder is a real task rather than a formatting exercise.

4.3 A larger benchmark, and what it says about us

The recorded cohort of Table 1 is 68 questions of a single shape. Our internal suite has the opposite problem: the strongest hosted models answer most of it perfectly, and a benchmark everyone passes ranks nothing [17, 23]. So we built a larger one and released it: 7,305 questions over 15 families and two environments, every answer computed from the simulator rather than annotated.

Three properties are worth stating because they are what make the set usable rather than merely large. Each family is *exactly* label-balanced by construction — items are bucketed by answer and drawn equally from each bucket, so a constant answer scores chance everywhere, and a family whose rare answer is genuinely rare yields fewer questions instead of being topped up. Every item is fingerprinted by `sha256` over its rendered state, question and sorted options, and checked against 97,133 training questions. And the same window is rendered three ways — an ASCII block, the identical cells as JSON, the identical cells as prose — with the surface drawn at random within each family, which makes the comparison between surfaces a randomised one.

The set was built to be hard, and on its first pass it was: `this-that-model-1.0` scored 0.409 against a chance rate of 0.343, and was *at* chance when the same maze window was written as JSON rather than as an ASCII block. We then generated 60,200 fresh questions from the same families — different items, different maze windows, checked by fingerprint *and* by rendered state so that nothing in the evaluation set could appear in training — and trained on them. Generating supervision that is at once diverse and structurally sound is a problem in its own right [3–5]; ours is the easy case of it, because the simulator that renders a state also computes the answer, so reliability is a property of the generator rather than something to be audited afterwards. The model now scores 0.839 on the full set and 0.844 on the per-family subset the hosted systems were measured

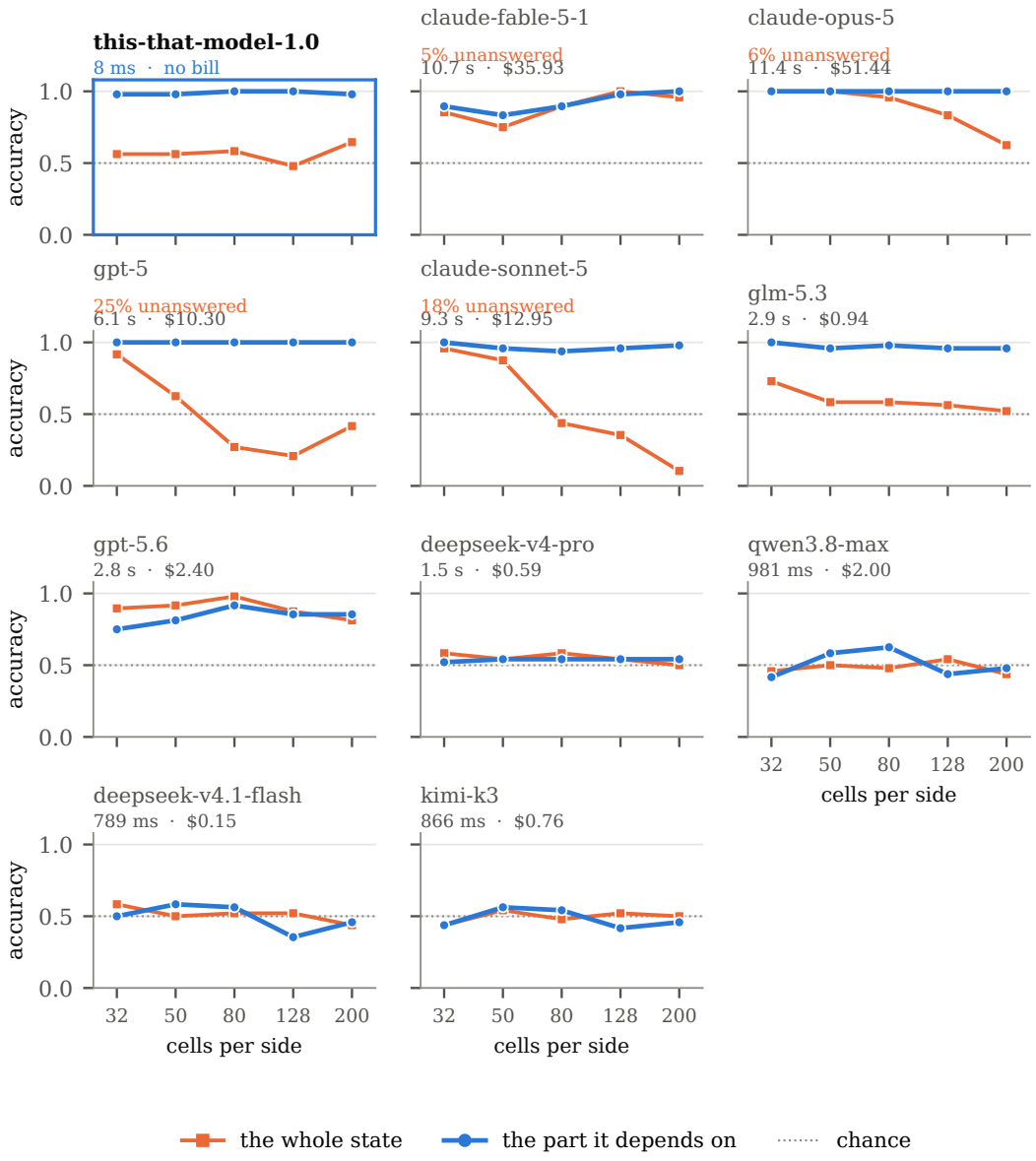


Figure 5. The same ladder, one panel per system, one shared scale, ordered by where the windowed curve ends up. Above each panel is the median time for one question and what the whole 480-question pass billed. Accuracy is over every question *asked*: a question a system did not answer is counted as wrong, and the orange line says what share of that system's questions those were. Scoring instead over the questions answered would condition on a subset each system selected using the variable this figure varies: the unanswered questions sit almost entirely on the whole-state arm and grow with the map, while the windowed arm of the same models has none at any size.

Table 3. The released benchmark, 2,250 questions drawn equally from every family.

system	tokens	accuracy	seen	unseen	no answer	cost ($)
gpt-5.6	286	0.897	0.947	0.879	42	7.38
claude-opus-5	510	0.892	0.908	0.886	141	109.26
this-that-model-1.0	0	**0.844**	0.873	0.834	0	free
claude-sonnet-5	722	0.836	0.792	0.853	278	29.12
Jev†	0	0.803	0.717	0.833	0	—
gpt-5	1212	0.800	0.700	0.837	365	28.20
glm-5.3	219	0.789	0.687	0.827	10	1.44
kimi-k3	7	0.659	0.750	0.626	0	0.52
qwen3.8-max	1	0.642	0.720	0.613	0	1.33
claude-fable-5-1	349	0.631	0.883	0.539	94	81.93
deepseek-v4.1-flash	89	0.512	0.605	0.478	0	0.21
deepseek-v4-pro	1	0.425	0.448	0.417	0	0.41
answering constantly	—	0.343	—	—	—	—

Every hosted system met these fifteen question shapes for the first time at test. this-that-model-1.0 was trained on all fifteen of them — on different items, over different maze windows, neither of which occurs in the evaluation set — so its row is an in-distribution number placed beside zero-shot ones, which is a real product fact and a weak scientific one. The honest reading is not that a 1.88B model overtook claude-sonnet-5: it is that a question shape you can generate training data for costs $0 and 30.9 ms per decision afterwards, and one you cannot stays where it was. *tokens* is the mean generated per question, and *no answer* is how many questions that system left unanswered, counted as wrong. †: Jev is the one row we did not measure — we hold no key for that endpoint and it was run for us on this same subset by a third party. It is the closest comparison here, being the only other system that answers with no generated tokens at all. Its *seen* column is lower than its *unseen* column; that split is by our training mixture and means nothing for anyone else, so for it the difference is only that those four families are harder.

on (Table 3), a factor of 2.05 over where it started.

What that training fixed, it fixed completely. The surface dependence is gone:

the state rendered as	before	after	chance
an ASCII block	0.457	0.779	0.388
the same cells as JSON	0.373	0.936	0.394
the same cells as prose	0.420	0.942	0.394

The JSON column began at chance and ends at 0.936. Whatever the model had learned before was a format; what it has now survives the format changing underneath it.

What the training did not fix is more interesting, and the benchmark localises it precisely. Of the fifteen families, 1500 questions — 21 % of the set — need a search over the map rather than a local or relative property: first_move (0.508), reachable_within (0.546), and distance_band (0.668), averaging 0.574. Every other family is between 0.77 and 0.998. The entire remaining gap is there.

The same quantity, asked two ways. Our four held-out families — shapes that appear in no training question anywhere — separate along the same line. same_line, which needs only relative geometry, reaches 0.980 against a chance rate of 0.333; reach_two, local counting, reaches 0.570. But distance_parity sits at 0.515 against 0.500, and plan_progress at 0.276 against 0.333. Both of those need a shortest path.

That comparison is across different mazes, which leaves an excuse: perhaps the parity questions landed on harder maps. So we removed it. On 300 mazes we asked for the shortest-path distance twice — once as one of five bands, the readout the model was trained on, and once as even or

odd, a readout it has never seen. The two questions are about the same number in the same maze. Where the band is right, the parity is right 0.513 of the time. A coin is 0.500.

The model is not computing the distance and reading it out two ways. It is recognising which band a picture belongs to, and when asked for any other function of the same quantity it has nothing to offer. A benchmark score on `distance_band` therefore does not mean what the family name suggests, and we would rather say so than let the number stand unqualified.

5 Conclusion

A decision is not a document. When a program needs to know which of n named things is the case, the useful reply is an index and a number, and every mechanism between the model's hidden state and that index — the decoding loop, the token budget, the parser, the retry — is machinery in the way. Removing it is what this paper is about, and the effect is arithmetic rather than architectural: 30.9 ms per decision, 32 decisions per second on one consumer card, no generated tokens, no bill, and no state leaving the machine. That is a component a program can put inside a loop, which a 8758 ms call is not.

Against `Jev`. The closest comparable system is `Jev` [1], which sells the same interface — declared options, a distribution over exactly those options — as a hosted endpoint. On the only inputs on which the two can be compared without either party choosing them, the 68 questions over 17 states that a third party recorded, we score 0.941 against its 0.765, with a Brier score of 0.042 against 0.133 (Table 1). We take that comparison seriously and state its limits: 68 questions is a small cohort, its wording is theirs rather than ours, and the accuracy gap of 18 percentage points rests on 12 questions.

`Jev` was run for us on the released benchmark of Section 4.3, on the identical 2,250-question subset every hosted system answered: it scores 0.803 against our 0.844. That comparison is the closest like-for-like in this paper — the same interface, the same declared options, no generated tokens on either side — and it is still not a clean one, because we trained on these fifteen question shapes and it had no reason to. What can be said without qualification is narrower: of the two systems selling this interface, ours is the one you can retrain on your own decision shapes, and doing so moved it from 0.409 to 0.844.

The more useful differences are not on that table. Jev publishes $0.042 per million input tokens with output free, and an end-to-end response time of 70–500 ms; we measured 30.9 ms end to end, below the bottom of that range, and our marginal cost is electricity on a card already bought rather than a price that must cover serving and margin — a difference of about one order of magnitude, not the five that separates us from a frontier LLM, and we say so in Figure 1 rather than letting the axis imply otherwise. The substantive difference is where the guarantee comes from. Both systems claim that a malformed answer cannot occur; ours follows from Equation (2), whose support is the declared option set, and theirs is asserted on the same structural grounds — their own write-up notes that the zero type-error figure is added to its plots because schema matching is guaranteed by construction rather than because it was measured. We agree with the reasoning. We also think a released checkpoint, an open inference stack and a published benchmark are what let a reader check it, and that is the form in which we are shipping this.

What the measurements actually support. Three claims survive our own attempts to break them. Decomposition is real and is not ours: on the context ladder every system that can do the task at all is flat and near-perfect given the part the answer depends on, and decays given the whole

map, with 19 % of the whole-map questions at the largest size returning nothing at all (Figure 5). Calibration is real and is rare: against a computed answer distribution we reach 0.750 where the ceiling is 0.746, while most hosted endpoints expose no probability at all and every one that does is worse than a constant predictor (Table 2). And generated tokens do not buy accuracy on decisions of this kind — on the stochastic-actuator family the systems that generate the most are the least accurate (Figure 2).

What does not. The benchmark we built and released began as a list of things this model could not do, and one round of targeted data removed most of it: 0.409 to 0.839, with the JSON rendering going from chance to 0.936. What survived that round is sharper for having survived it. 1500 of the questions need a search over the map, and they sit at 0.574 while everything else is above 0.77. Asked for the same shortest-path distance as a band and as a parity, the model answers the band and then flips a coin on the parity, at 0.513 where a coin is 0.500 — so the band is a label it recognises, not a number it holds. That is what a single forward pass through a fixed stack of layers can and cannot be made to do, and no amount of data of that shape moved it.

We would rather a reader leave with the narrow claim than the wide one. A frontier model is a better judge and a worse component; the gap between them is paid in latency and tokens rather than in intelligence; this model will answer a bounded question about a state in 30.9 ms for the cost of the electricity, and will not search a graph for you at any price. A great many branches in real software need only the bounded question; the ones that need a search should be given a search.

Availability

`this-that-model-1.0` is released on the Hugging Face Hub at `huggingface.co/flock-io/this-that-model-1.0` under the MIT licence, with the inference package and the scripts that reproduce every measurement in this paper at `github.com/FLock-io/this-that-model`. The weights load into the typed-decision stack described in Section 3.1 — on CUDA, on Apple Silicon or on CPU — so an existing caller changes a model name and nothing else. It is adapted from `decider-2b`, a publicly released 2B typed-decision checkpoint and inference stack, under Apache-2.0. The environment simulator, the stochastic-actuator construction behind Equation (12), and the recorded hosted-service cohort of Table 1 come from NanoJev under the MIT licence; we use them as published, unmodified, and thank their authors. The benchmark of Section 4.3 — 7,305 questions over 6,525 distinct states, with the generators, the balancing and the contamination fingerprints — is released on the Hugging Face Hub under the MIT licence, at `huggingface.co/datasets/limberc/this-that-spatial-bench`. The internal suite is an instrument, not a release. Correspondence: `ai@flock.io`.

References

[1] Diogo Almeida. Introducing system one models & jev. TypeSafe AI blog, September 2026. URL https://typesafe.ai/blog/introducing-system-one-models-and-jev. Published 15 September 2026.

[2] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

[3] Zehua Cheng, Wei Dai, Jiahao Sun, and Thomas Lukasiewicz. CausalSynth: Generating structurally sound synthetic data. *arXiv preprint arXiv:2605.17528*, 2026. URL https://arxiv.org/abs/2605.17528.

[4] Zehua Cheng, Wei Dai, Jiahao Sun, and Thomas Lukasiewicz. CircuitSynth: Reliable synthetic data generation. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026. arXiv:2604.10114.

[5] Zehua Cheng, Wei Dai, Jiahao Sun, and Thomas Lukasiewicz. GraphSynth: Resolving the diversity-reliability trade-off with probabilistic factor graphs. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 42527–42543. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1969. URL https://aclanthology.org/2026.acl-long.1969/.

[6] Samantha R. Cook, Andrew Gelman, and Donald B. Rubin. Validation of software for Bayesian models using posterior quantiles. *Journal of Computational and Graphical Statistics*, 15(3):675–692, 2006.

[7] Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 8-bit optimizers via block-wise quantization. *International Conference on Learning Representations (ICLR)*, 2022.

[8] Ran El-Yaniv and Yair Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11:1605–1641, 2010.

[9] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[10] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[11] Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

[12] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, 2017.

[13] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[14] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023.

[15] Daniel Kahneman. Thinking, fast and slow. 2011.

[16] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. *International Conference on Machine Learning (ICML)*, 2020.

[17] Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, Zhiyi Ma, Tristan Thrush, Sebastian Riedel, Zeerak Waseem, Pontus Stenetorp, Robin Jia, Mohit Bansal, Christopher Potts, and Adina Williams. Dynabench: Rethinking benchmarking in NLP. In *Proceedings of NAACL-HLT*, pages 4110–4124, 2021.

[18] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. Efficient memory management for large language model serving with PagedAttention. *Symposium on Operating Systems Principles (SOSP)*, 2023.

[20] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In *AAAI Conference on Artificial Intelligence*, 2015.

[21] Jeremy Nixon, Michael W. Dusenberry, Linchuan Zhang, Ghassen Jerfel, and Dustin Tran. Measuring calibration in deep learning. In *CVPR Workshops*, 2019.

[22] OpenJev contributors. Nanojev: an open reproduction of the typed-decision interface. Hugging Face model repository, MIT licence, 2026. URL https://huggingface.co/C-Tianyu/NanoJev.

[23] Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023.

[24] Sean Talts, Michael Betancourt, Daniel Simpson, Aki Vehtari, and Andrew Gelman. Validating Bayesian inference algorithms with simulation-based calibration. *arXiv:1804.06788*, 2018.

[25] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv:2307.09702*, 2023.

[26] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning (ICML)*, 2022.

[27] Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[28] An Yang et al. Qwen3 technical report. *arXiv:2505.09388*, 2025.

[29] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Gated delta networks: Improving mamba2 with delta rule. *International Conference on Learning Representations (ICLR)*, 2025.